Mellum

Slides · aral.cc/mellum2.1.pdf

Mellum 2: Cooking an Agentic Code Model

Mellum2 Technical Report[1] + Bonus Content

Mellum team, JetBrains

1 Kojic et al., Mellum2 Technical Report (JetBrains, 2026) — arxiv.org/abs/2605.31268

Mellum Team

Aral de Moor	Evaluation Lead
Marko Kojic	Pre-Training Lead
Ivan Bondyrev	Post-Training Lead
Bulat Salimzianov	RL Lead

\+ Joseph Shtok Petr Borovlev Kseniia Lysaniuk Madeeswaran Kannan
Ivan Dolgov Nikita Pavlichenko

Two Goals

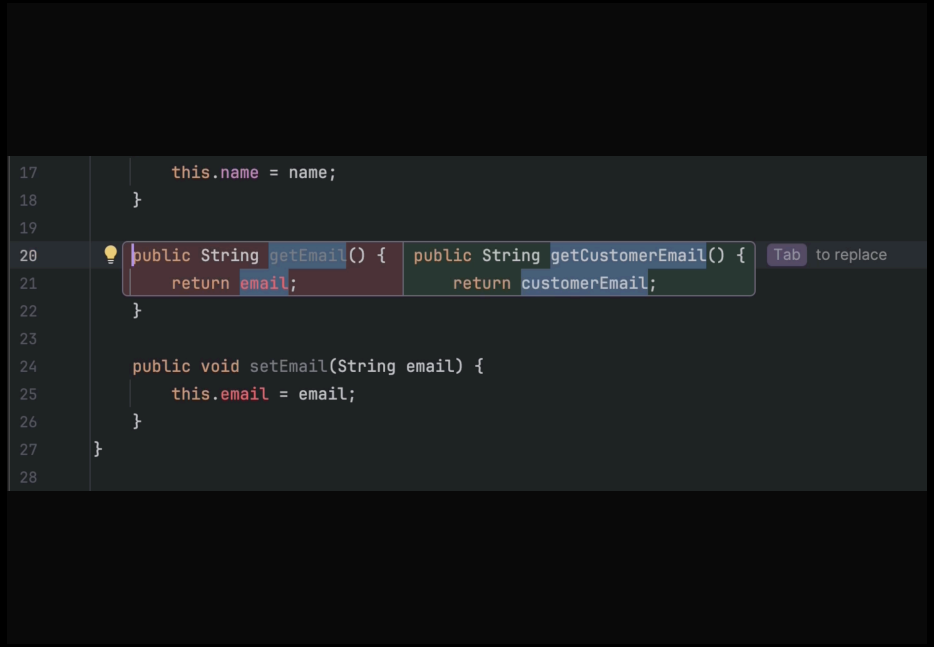
01

Ship the best model for next-edit suggestion

cheap, fast, and good

02

Try to make it agentic



Why Cook up a 12B-A2.5B Code Model?

It’s a pretty expensive endeavour

Production use	Latency, data governance, best possible code completions.
Strategic open source	Can we do better than Qwen2.5?
Research	Several JetBrains labs experiment with our models in novel contexts.

Outline



00 Pre-Training
01 Fine-Tuning
02 Reinforcement Learning
03 Results
04 What's Next

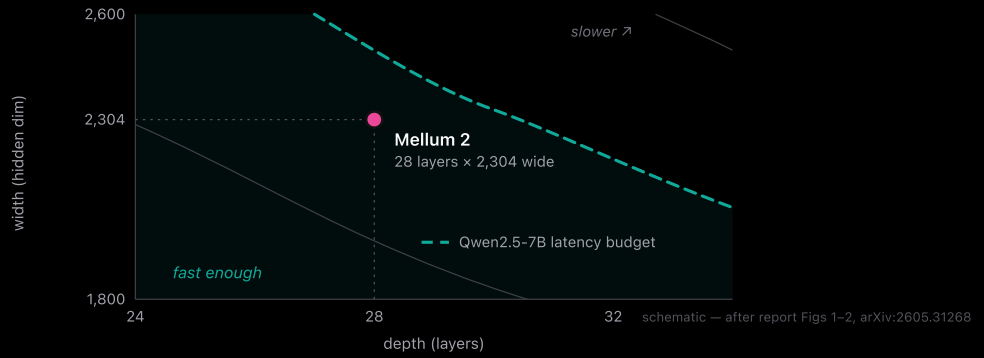
00

Pre-Training

Pre-Training: Design Approach

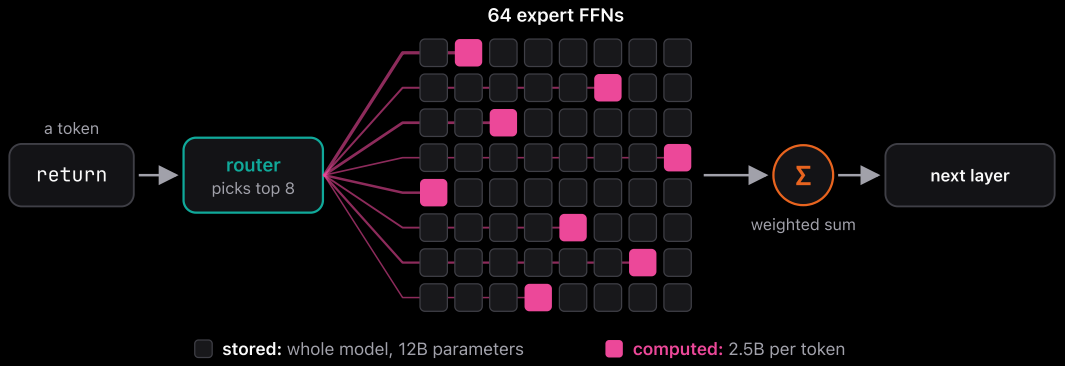
Optimised for next-edit suggestion (at the time): 2k context in, short edit out

1. Constraint: match Qwen2.5-7B on one H100 (FP8), latency and saturated throughput
2. Architectural ablations: dense vs MoE, active experts, KV heads, attention, MTP
3. Per candidate: iso-latency maps find in-budget configs, then assess quality[1]



Pre-Training: Mixture of Experts (MoE)

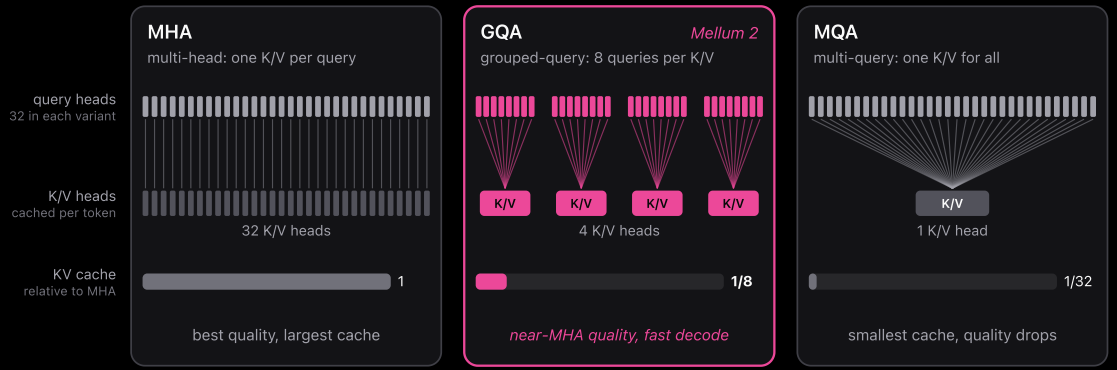
- MoE layer: many expert FFNs; a router sends each token to k[2,3]
- Mellum 2: 64 experts, 8 active, no shared expert (Qwen3-MoE)[1,4]
- Why: 12B of capacity at the per-token cost of 2.5B dense[1]



2 Shazeer et al., Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer — arxiv.org/abs/1701.06538
3 Fedus et al., Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity — arxiv.org/abs/2101.03961
4 Yang et al., Qwen3 Technical Report — arxiv.org/abs/2505.09388

Pre-Training: Grouped-Query Attention (GQA)

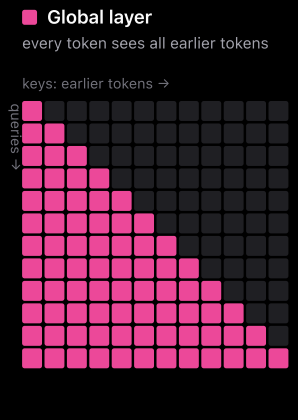
- Query heads share K/V heads in groups, between MHA and MQA[5,6]
- Decoding is memory-bound: fewer K/V heads, smaller cache, faster tokens
- Mellum 2: 8 queries per K/V head; 8 K/V heads cost speed, 2 cost quality[1]



5 Shazeer, Fast Transformer Decoding: One Write-Head is All You Need — arxiv.org/abs/1911.02150
6 Ainslie et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints — arxiv.org/abs/2305.13245

Pre-Training: Sliding-Window Attention (SWA)

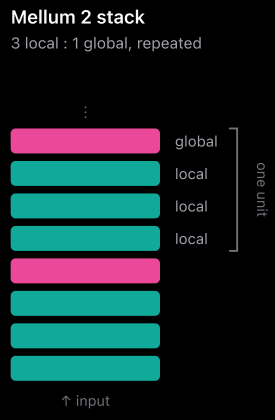
- Sliding-window attention: each token sees only the last W tokens[7,8]
- Mellum 2: 3 local layers per global layer, window from Gemma 3[1,9]
- Local layers cap KV cache and prefill; global layers keep long range[1]



KV cache grows with context
long-range access



KV cache capped at W tokens
prefill cost linear in length

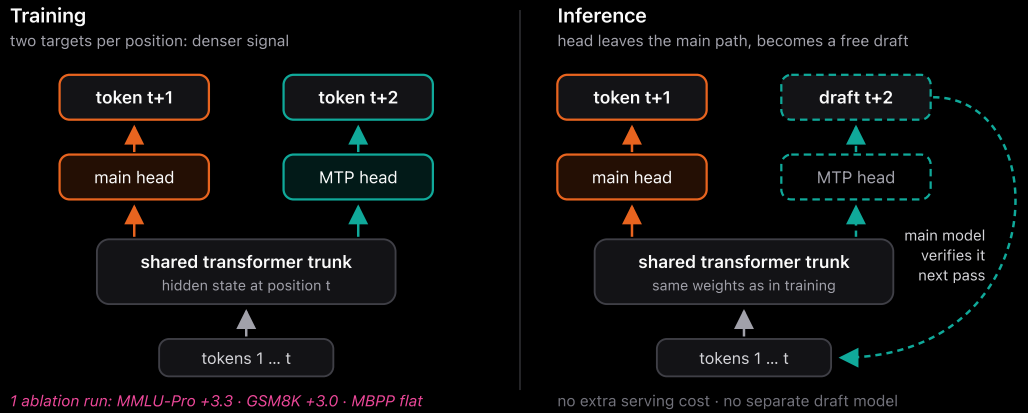


most layers: bounded cost
globals keep long range

7 Beltagy et al., Longformer: The Long-Document Transformer — arxiv.org/abs/2004.05150
8 Jiang et al., Mistral 7B — arxiv.org/abs/2310.06825
9 Gemma Team, Gemma 3 Technical Report — arxiv.org/abs/2503.19786

Pre-Training: Multi-Token Prediction (MTP)

- An extra head also predicts token t+2: two targets per position[10,11]
- Mellum 2 keeps one low-weight head; single ablation: MMLU-Pro +3.3, GSM8K +3.0, MBPP flat[1]
- No serving cost: the head becomes a speculative-decoding draft[12]



1 ablation run: MMLU-Pro +3.3 · GSM8K +3.0 · MBPP flat

no extra serving cost · no separate draft model

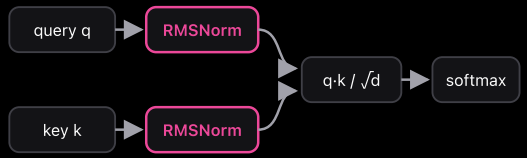
10 Gloeckle et al., Better & Faster Large Language Models via Multi-token Prediction — arxiv.org/abs/2404.19737
11 DeepSeek-AI, DeepSeek-V3 Technical Report — arxiv.org/abs/2412.19437
12 Leviathan et al., Fast Inference from Transformers via Speculative Decoding — arxiv.org/abs/2211.17192

Pre-Training: QK-Norm

- Normalize each head's queries and keys (RMSNorm) before the dot product[13,14]
- Stops logits growing until softmax goes one-hot and loss spikes[14,15]
- Mellum 2 inherits it from its Qwen3-MoE-style decoder, for stability[1]

Where the norm sits

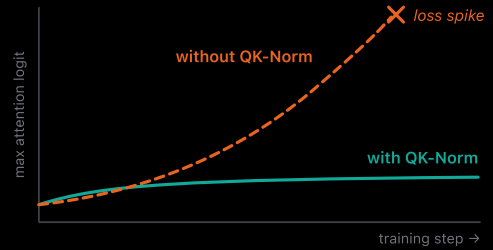
per attention head, over the head dimension



logit scale is set by a learned gain, not by growth in the Q/K weights

What it prevents

illustrative schematic, not Mellum data



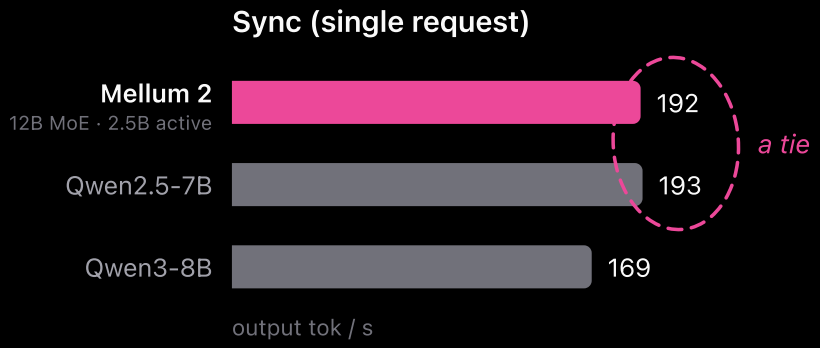
without: logits grow → softmax goes one-hot → loss spikes

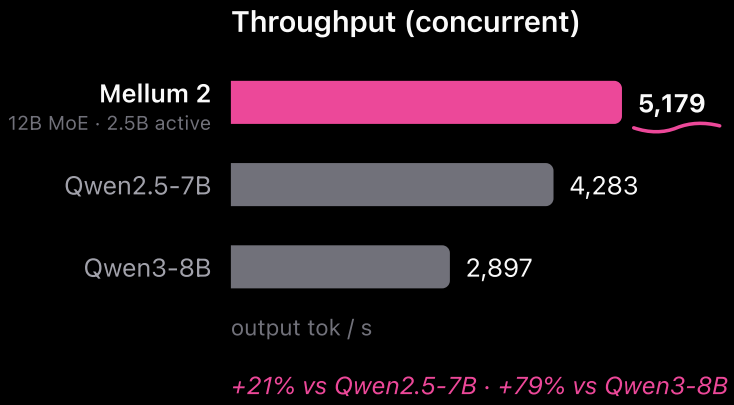
with: logits stay bounded → attention stays spread

13 Henry et al., Query-Key Normalization for Transformers — arxiv.org/abs/2010.04245
14 Dehghani et al., Scaling Vision Transformers to 22 Billion Parameters — arxiv.org/abs/2302.05442
15 Wortsman et al., Small-scale proxies for large-scale Transformer training instabilities — arxiv.org/abs/2309.14322

Pre-Training: Result in Tokens per Second

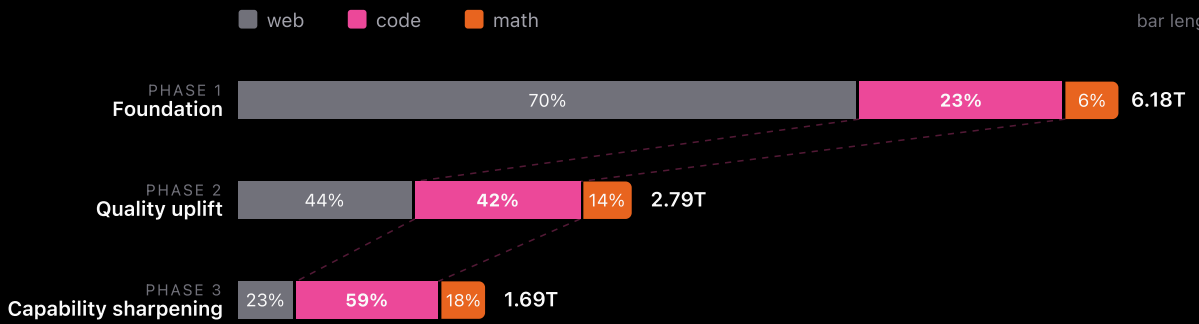
- Next-edit shape: long context in, short edit out (2,304 / 256 tokens)
- Single request: on par with Qwen2.5-7B (192 vs 193 tok/s)
- Saturated: +21% throughput over Qwen2.5-7B, +79% over Qwen3-8B





Pre-Training: Data

- 10.65T tokens; Muon[16,17] + BF16/FP8 hybrid throughout; FIM[18]
- Data: permissively licensed repo code, Common Crawl[19] web, synthetic rewrites, math textbooks (§3.1); derived from Nemotron-CC[20] and FineWeb[21]

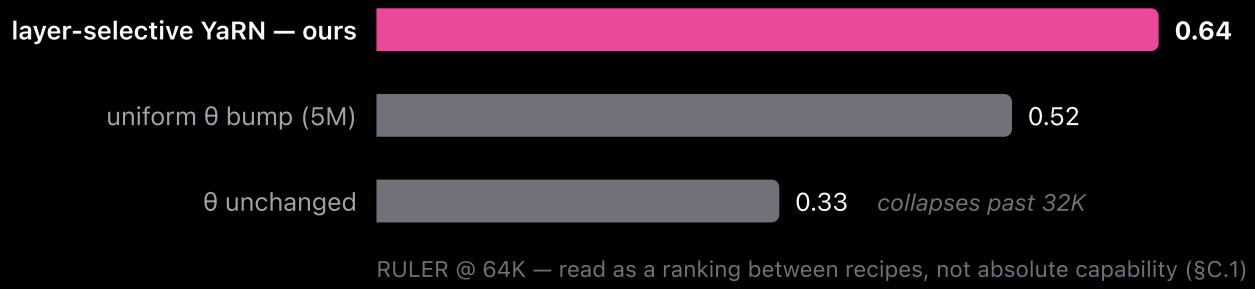


code share climbs 23% → 42% → 59%; FIM rate 50% → 10% → 50% (phase-3 FIM on code only)

16 Jordan et al., Muon: An optimizer for hidden layers in neural networks (2024) — kellerjordan.github.io/posts/muon
17 Liu et al., Muon is Scalable for LLM Training (Moonlight) — arxiv.org/abs/2502.16982
18 Bavarian et al., Efficient Training of Language Models to Fill in the Middle — arxiv.org/abs/2207.14255
19 Common Crawl — commoncrawl.org
20 Su et al., Nemotron-CC — arxiv.org/abs/2412.02595
21 Penedo et al., The FineWeb Datasets — arxiv.org/abs/2406.17557

Pre-Training: Long Context

- Layer-selective YaRN[22], global layers only — recipe from Gemma 3[9] and OLMo 3[23]
- Kept training past saturation just so the router could settle
- Negative: adding OLMo 3's Longmino (long science PDFs + synthetic tasks) cost 2–3pp RULER everywhere[23]



§C.1: absolute RULER[24] depressed by a QA prompt-formatting issue — read as within-recipe ranking.

22 Peng et al., YaRN: Efficient Context Window Extension of LLMs — arxiv.org/abs/2309.00071
23 Team OLMo, Olmo 3 — arxiv.org/abs/2512.13961
24 Hsieh et al., RULER: What's the Real Context Size of Your Long-Context LMs? — arxiv.org/abs/2404.06654

01

Fine-Tuning

Fine-Tuning: SFT vs RLVR

SFT

curated demonstrations → imitate, token by token

RLVR

sample answers → a program checks them → reinforce what passes

Fine-Tuning: Data Mix

TLDR: 👀 **look** at the data



Mix

chat & IF | coding | agentic trajectories | tool use + error recovery | reasoning | safety

identity ×3

Fine-Tuning: Identity

- No Google data in the mix
- Early runs: the model introduced itself as a Google assistant
- Fix: 3× oversampling of identity dialogues



zero Google models in the synthetic-data pipeline. still.

02

Reinforcement Learning

RL: Fast Iteration Is the Point

10k

SWE sandboxes running concurrently

2 days

for a decent async GRPO run
= 1,000,000s of sandboxes

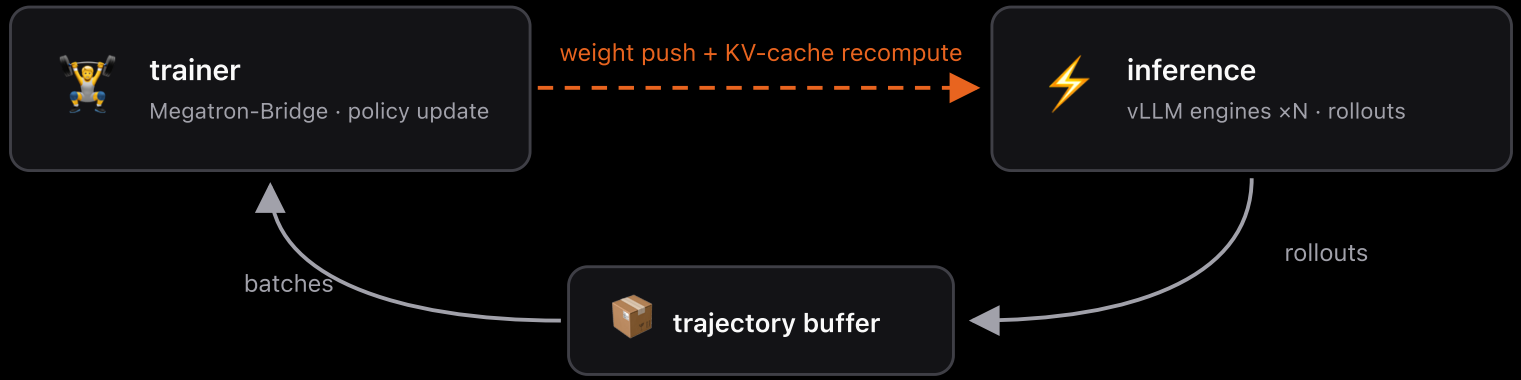
700

training steps per run

CPU verification is fully decoupled: SWE sandboxing scales independently of the GPUs

RL: Infrastructure

- Agentic rollouts vary in length; a synchronous loop waits for the slowest
- Asynchronous: generate and train at once; rollouts never wait for gradients



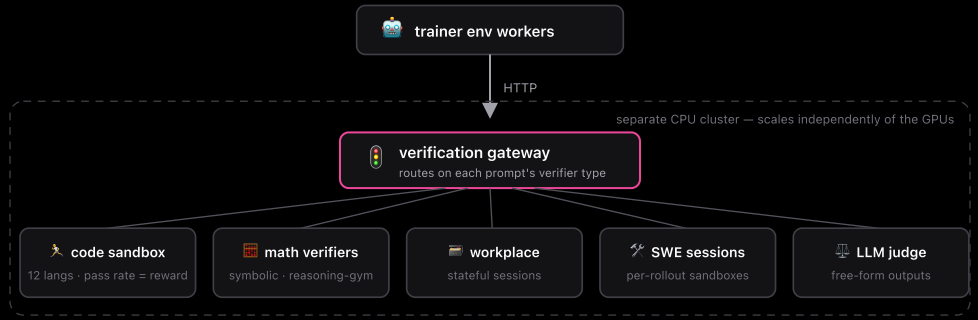
rollouts never wait for gradients — a trajectory may span two consecutive policy versions (staleness ≤ 2 steps)

Redrawn from report Fig 10[1]; NeMo-RL[33].

33 NVIDIA NeMo-RL (async GRPO) — github.com/NVIDIA-NeMo/RL

RL: Infrastructure

- Checking answers is mostly CPU work: run tests, call solvers, replay sessions
- A gateway routes each answer to its verifier, on a separate cluster[1]
- 2.1 IdeGYM[38] runs tens of thousands of environment pods in parallel
- 2.1 verifiers on a CPU cluster that scales independently of the GPUs
- 2.1 software engineering (SWE): real repos, shell and file tools; tests decide reward[31]



Redrawn from report Fig 11[1] + the verifier architecture; reasoning-gym[39].

38 JetBrains Research, IdeGYM — github.com/JetBrains-Research/idegym
39 Stojanovski et al., Reasoning Gym (NeurIPS 2025 D&B) — arxiv.org/abs/2505.24760

RL: Recipe (GRPO)

Reward = fraction of tests passed, so a partial fix still scores

If all 16 answers score the same, none beats the others: no gradient, drop the group[27,28,30]

Unscoreable answer: zero reward, and the error message comes back next rollout

Verifier crashes are retried, never scored, so the reward signal stays clean

27 Shao et al., DeepSeekMath (introduces GRPO) — arxiv.org/abs/2402.03300
28 Ahmadian et al., Back to Basics: Revisiting REINFORCE Style Optimization (RLOO) — arxiv.org/abs/2402.14740

RL: Recipe

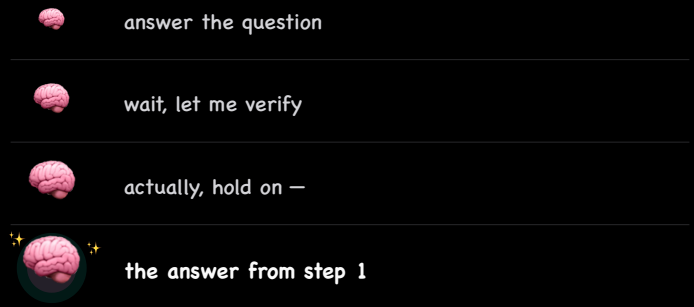
Don't divide by the group's spread (2.0 and 2.1): it overweights near-solved and hopeless prompts[29]

Average the loss over tokens, not answers, so long answers aren't under-trained[30]

`2.1` RL grows from a short final stage into the main training stage[31]

`Instruct` reward shaping penalises any visible thinking: answer directly

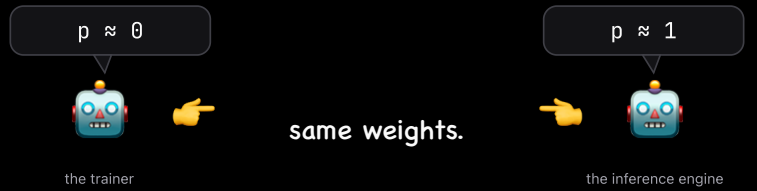
`Thinking` reward shaping penalises overlong reasoning and answers that skip tools[41]



29 Liu et al., Understanding R1-Zero-Like Training: A Critical Perspective (Dr. GRPO) — arxiv.org/abs/2503.20783
30 Yu et al., DAPO: An Open-Source LLM RL System at Scale — arxiv.org/abs/2503.14476
31 JetBrains, Mellum2.1 Thinking model card (Oct 2026) — huggingface.co/JetBrains/Mellum2.1-12B-A2.5B-Thinking
41 Yu et al., Stop Unnecessary Reflection (ARLCP) — arxiv.org/abs/2602.12113

RL: Training–Inference Mismatch

- Industry-wide, not a Mellum quirk: kernel sum order changes with batch size[34,35]
- Yao et al.[36]: same weights, vLLM said p≈1, the trainer p≈0 — silently off-policy
- MoE makes it worse: the two engines route to different experts, plus BF16 rounding
- Fix, 2.0 and 2.1: mask tokens whose trainer/inference ratio leaves [0.5, 5.0][37]
- **Mask, don't clip: after an expert flip the ratio carries no signal (instead of rollout-replay)**



34 Thinking Machines, Defeating Nondeterminism in LLM Inference (Sep 2025) — thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference
35 LMSYS, deterministic inference in SGLang (Sep 2025) — lmsys.org/blog/2025-09-22-sglang-deterministic
36 Yao et al., rollout-training mismatch (NeurIPS'25 OPT-ML) — fengyao.notion.site/off-policy-rl
37 Ling Team, Every Step Evolves: Scaling RL for Trillion-Scale Thinking Model (Ring-1T, IcePop) — arxiv.org/abs/2510.18855

RL: Training-Inference Mismatch

Rollouts come from an older policy, so each token is reweighted by new/old

Standard clipping: once that ratio leaves [0.8, 1.2], the token gets no gradient

2.0 clip-higher: raise only the upper bound to 1.28[30]

2.1 CISPO: clip the weight, not the token, so every token keeps its gradient[32]

Rare tokens (“wait”, “recheck”) hit the bound first; with CISPO they keep learning

32 MiniMax, MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention (CISPO) — arxiv.org/abs/2506.13585

03

Results

Results: Instruct

Benchmark	Mellum 2 Instruct	Qwen3.5-4B	OLMo-3-7B
LiveCodeBench v6	37.2	**51.0**	28.2
EvalPlus	**78.4**	69.4	67.3
BFCL v4	44.2	**52.0**	19.8
BFCL v3	**66.3**	64.1	41.9
IFEval	75.8	82.1	**83.2**
AIME 25/26	**41.7**	38.3	40.0

Bold = best in row. Mellum 2 Instruct after RL, report pipeline: greedy, temp 0 except BFCL 0.01, LCB 0.2.
Source: report Table 9[1].

Results: Thinking

Close to Qwen3.5-9B, at **~2×** its throughput (1.3× single-request)

Benchmark	Mellum 2.1	Mellum 2	Gemma 4 E4B	Qwen3.5-9B
LiveCodeBench v6	**82.0**	69.4	69.4	75.4
SWE-bench Verified	47.0	2.0	23.0	**50.0**
SWE-bench Pro	28.0	0.0	4.0	**38.0**
Terminal-Bench 2.1	17.4	0.6	3.4	**21.7**
BFCL v4	**62.3**	49.6	52.5	58.5
IFEval	90.6	79.5	90.8	**92.4**
AIME 25/26	83.3	60.1	45.0	**86.7**

Bold = best in row. Model-card pipeline, self-reported by JetBrains; every model re-run, Mellum 2 included. Agentic rows: same harness (Pi v0.73.1, shell + file tools, 114K context). Source: model card[31].

04

What's Next

What's Next: Small SWE Agents

Jevons paradox: cheaper use of a resource raises total consumption[42]

Cheaper, faster small agents mean more agent usage, so efficiency matters

Open problem: frontier models barely need a scaffold (bash-only)[43,44]

42 W. S. Jevons, The Coal Question (1865); see Jevons paradox — en.wikipedia.org/wiki/Jevons_paradox
43 mini-swe-agent, SWE-bench bash-only leaderboard — github.com/SWE-agent/mini-swe-agent · swebench.com (“Bash Only” view)
44 DeepSeek-AI, DeepSeek-V4 (evals use bash + file-edit only) — arxiv.org/abs/2606.19348

Model and Checkpoints

Checkpoints on HuggingFace (Apache 2.0)

Base-Pretrain	JetBrains/Mellum2-12B-A2.5B-Base-Pretrain
Base	JetBrains/Mellum2-12B-A2.5B-Base
Instruct-SFT	JetBrains/Mellum2-12B-A2.5B-Instruct-SFT
Thinking-SFT	JetBrains/Mellum2-12B-A2.5B-Thinking-SFT
Instruct RL	JetBrains/Mellum2-12B-A2.5B-Instruct
Thinking RL	JetBrains/Mellum2-12B-A2.5B-Thinking

What's Next: Try It



huggingface.co/JetBrains

All checkpoints, Apache 2.0:

`Mellum2-12B-A2.5B-Base`
`Mellum2-12B-A2.5B-Base-Pretrain`
`Mellum2-12B-A2.5B-Instruct-SFT`
`Mellum2-12B-A2.5B-Instruct`
`Mellum2-12B-A2.5B-Thinking-SFT`
`Mellum2-12B-A2.5B-Thinking`
`Mellum2.1-12B-A2.5B-Thinking (+ GGUF)`

the RL checkpoints are the bare Instruct / Thinking names

Runs locally: `llama.cpp` (Ollama / LM Studio) — official GGUFs, BF16 → Q4_K_M

Mellum

Get in Touch :)))

Theses · internships · collaborations

mellum@jetbrains.com

Questions welcome